Drone Security Analyst Agent

FlytBase AI Engineer Assignment

Abdul Basith Syed

Technical report, 30 September 2026

Implementation frozen at repository commit `61bc003`

> This report describes what was built, how it was evaluated and what the evaluation does and does not show. Every number is taken from the repository's committed documentation, the evaluation snapshot in `docs/evaluation.md` (regenerated by an offline, deterministic harness) or the recorded real-video run in `docs/real_video_experiment.md`. Results from controlled synthetic fixtures and results from the real-video run are labelled separately throughout: Controlled / synthetic · Real video · Automated tests · Measured on this machine

Contents

1. Executive summary
2. Problem definition and assumptions
3. System architecture
4. Data and memory design
5. Vision and video pipeline
6. Event and entity reasoning
7. Security detection rules
8. Analyst agent
9. Experimental evaluation
10. Real-video experiment
11. Scalability
12. Testing
13. Limitations and threats to validity
14. Future work (not implemented)
15. AI-assisted development
16. Demo and submission materials
17. Conclusion

Appendix A. Assignment requirement coverage

1. Executive summary

What was built. A Python prototype of a drone security analyst for a fixed property. It takes scripted text-frame scenarios or real video, joins each frame to *simulated* telemetry, records what is seen as observations in a SQLite frame-by-frame index (with FTS5 search), merges repeated detections into temporal events, groups separate vehicle events into coarse "similar vehicle" entities, applies eight deterministic rules (four over camera events, four over drone telemetry) to produce evidence-linked alerts with severity ladders, recommended actions and a lifecycle, and lets an analyst answer questions and summarise the run with explicit citations to stored evidence. It is driven from a command-line interface (a scripted patrol can also run progressively, raising alerts tick by tick), a small offline dashboard shows the stored evidence, and an offline, deterministic evaluation harness measures its components.

Core architecture. `FRAME → OBSERVATION → EVENT → MEMORY → REASONING`. Frames are never judged in isolation: observations are aggregated into events with duration, vehicle events are linked into coarse entities, and rules reason over events and entities. A frame-change gate avoids sending near-identical frames to the vision model, and a cassette layer records and replays model responses so that demonstrations and tests are reproducible without a key.

Deterministic rules versus LLM reasoning. Alerts, severities, events and entities are produced only by deterministic code with configured thresholds. An LLM (Gemini through LangChain) can be used in two places: as an optional vision model that describes frames, and as an optional analyst that explains stored evidence. It has no write access, cannot create, suppress or re-rate an alert, and its answers are shown only if a validator confirms that every cited id exists in the database and was actually retrieved. A deterministic template analyst works with no key and is the fallback.

What makes the approach useful. Every alert and every answer traces back to event, observation and frame identifiers; uncertainty is stated in the wording ("similar vehicle", never identity); simulated telemetry, placeholder confidence and truncated lists are labelled where they appear; and each component has an ablation or a stated limitation.

> **How to read the results.** Controlled synthetic experiments (Section 9) measure whether the pipeline behaves as designed on authored fixtures. The single real-video run (Section 10) validates end-to-end integration with real Gemini output on one short clip; it has no perception ground truth. Neither establishes real-world detection accuracy, and this report makes no such claim.

2. Problem definition and assumptions

A docked drone patrols one property. Nobody watches hours of footage, so the agent must turn drone video and telemetry into a short list of things that matter (alerts), a searchable history, and answers that show their evidence. The assignment asks for simulated telemetry and video frames, object and event logging with context, rule-based alerts, and a frame-by-frame indexing component, plus a feature specification, architecture, tests, a report and demonstration videos (coverage table in Appendix A).

Explicit assumptions

Area	Assumption
Property	One fixed property, one drone, one patrol per run; six zones: `main_gate`, `driveway`, `parking_area`, `warehouse`, `restricted_zone`, `garage`.
Telemetry	**Simulated**: a seeded simulator (or a timeline you supply) produces position, altitude, battery and zone. A video carries none. An object's zone is the drone's zone when the frame was captured.

Area	Assumption
Time	Video time is mapped onto a simulated wall clock, so time-of-day rules (for example crossing midnight) work on any clip. “Real time” means streaming aggregation on this simulated clock, not a live feed.
Observations	Scripted scenarios use authored text descriptions as deterministic ground truth for tests; real video is described by a vision model behind the same provider interface. Model output is treated as noisy.
Identity	No face recognition and no personal identification. Vehicles are grouped only by coarse attributes (colour, body type) and always described as “similar”; people are never linked across events.
Access	The core pipeline, tests and offline demonstrations need no API key or network. Gemini is optional and configured through environment variables only.
Scope	Explicitly out of scope: embeddings or vector search, YOLO or multi-object tracking, multi-drone, real MAVLink/RTSP integration, face recognition, Docker/CI, LangGraph. (The local read-only dashboard, battery and geofence rules and a 10K/100K benchmark were first postponed and later built.)

3. System architecture

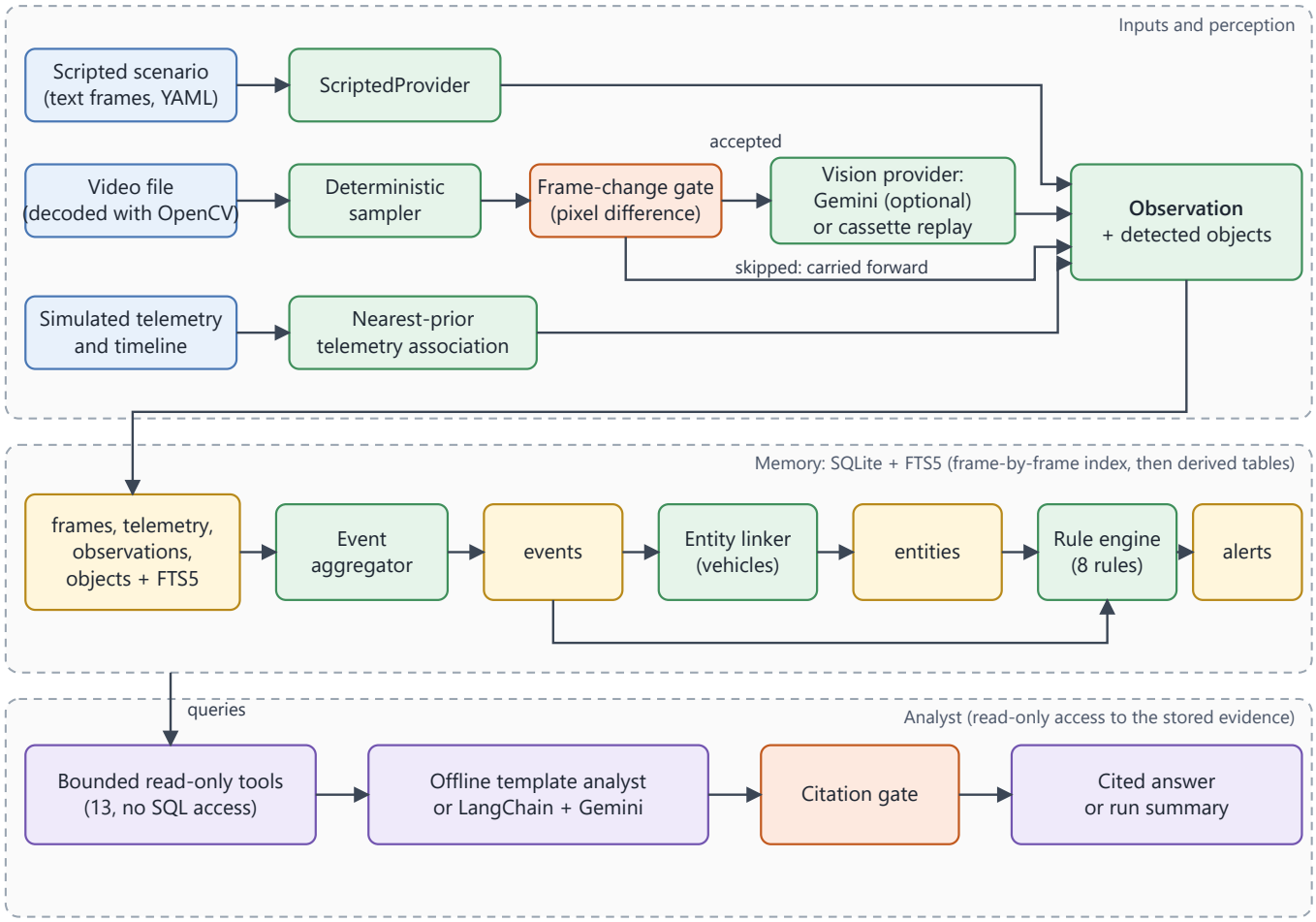


Figure 1. Current architecture. Green boxes are deterministic processing, yellow boxes are stored data, orange boxes are gates, purple boxes belong to the analyst. Gemini appears only as an optional vision provider and an optional analyst backend.

Stages and where they live

Stage	Responsibility	Package
Scenario / video	Scripted text-frame scenarios, or a real video decoded with OpenCV, sampled every N seconds and gated by a pixel-difference check.	`scenario.py`, `video/`
Vision provider	`VisionProvider.analyze(frame)` is the pipeline boundary; a real model implements the smaller `FrameDescriber.describe(frame)`. Providers: scripted, cassette (record/replay), Gemini.	`vision/`
Telemetry association	Nearest-prior sample joined to each frame; stale when older than a threshold.	`telemetry/`

Stage	Responsibility	Package
Observation index	SQLite tables and an FTS5 index; every derived table links back to observations and frames.	`store/`
Event aggregation and entity linking	Repeated detections become events with duration; separate vehicle events form coarse entities.	`pipeline/`
Rule engine and alert lifecycle	Eight deterministic rules with YAML thresholds (four camera, four telemetry-safety) produce alerts with evidence, severity ladders, recommended actions and a status history (open, escalated, acknowledged, resolved).	`rules/`
Streaming runtime	`run` consumes telemetry and scripted frames tick by tick: an adaptive sampler decides which frames need a perception call, a perception agent (timeout, fallback, guard) returns observations, the context engine updates events and entities, and rules run after every tick so alerts are saved and printed as they fire.	`runtime/`, `agents/`
Analyst	Thirteen read-only tools, a template analyst, an optional LangChain + Gemini loop, a citation gate.	`agent/`
Evaluation harness	Offline, deterministic experiments and an ablation table; production code does not depend on it.	`eval/`

Trust boundary. Rules decide; the LLM explains and answers from tool output only. LangChain is imported in one file (`agent/llm.py`); the video, vision, pipeline, rules, store, model and telemetry packages never import it, and an automated test enforces this, so the deterministic pipeline runs without the LangChain extra.

4. Data and memory design

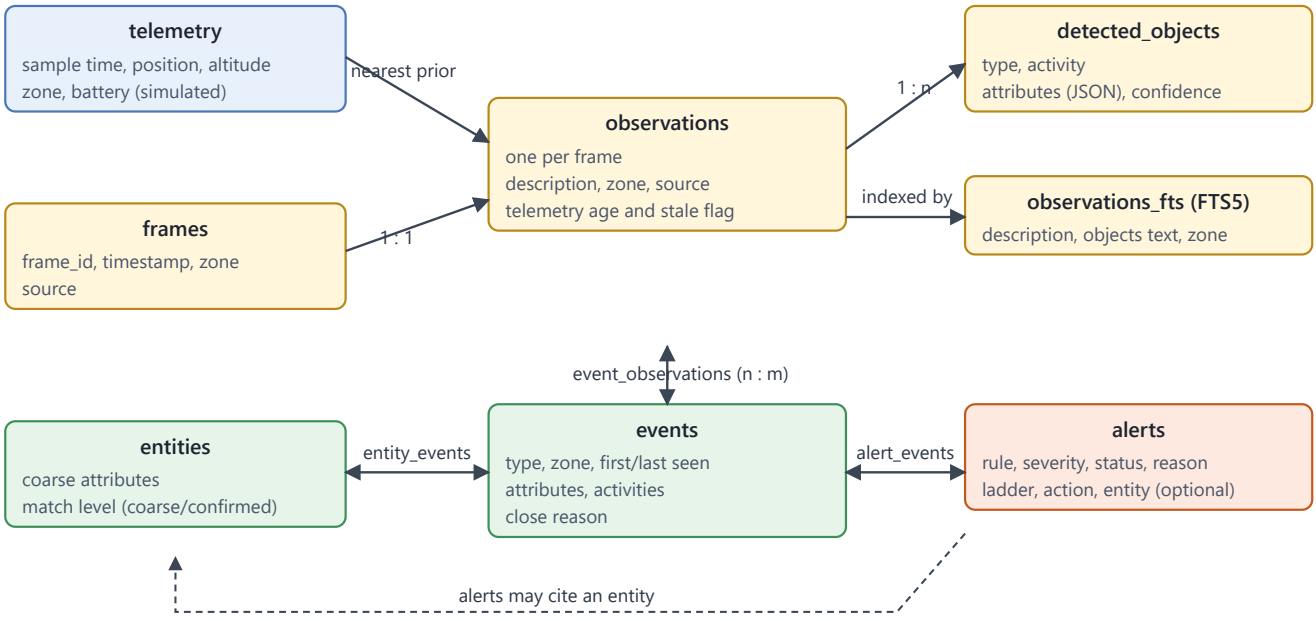


Figure 2. Stored data and evidence relationships (core tables; the current schema is version 5). Links are enforced with SQLite foreign keys.

Tables

Table	Purpose
`telemetry`	Simulated telemetry samples (time, position, altitude, zone, battery) with range checks.
`frames`	Frame-level index: frame id, timestamp, source and zone.
`observations`	One per frame: description, zone, source (`scripted`, `gemini:<model>`, `cassette`, or the same with `+carried`), and the telemetry association with its age and stale flag.
`detected_objects`	Objects in an observation: type (person, vehicle, other), activity, coarse attributes as canonical JSON, and confidence.
`observations_fts`	FTS5 index over description, an objects text (type, activity, attribute values) and the zone name; tokenizer `porter unicode61`.

Table	Purpose
`events`, `event_observations`	Derived events (type, zone, first/last seen, attributes, activities, close reason) and their ordered evidence links.
`entities`, `entity_events`	Derived coarse vehicle entities and their member events.
`alerts`, `alert_events`	Rule output: rule, severity, status, reason, the numbers the rule compared, confidence, verdict, recommended action, severity ladder, triggering events and optional entity.
`alert_history`, `alert_telemetry`	The lifecycle (raised, escalated, acknowledged, resolved, with who and when) and the telemetry samples that support a drone-safety alert.
`scenarios`, `agent_runs`	Which scenario a streaming run processed, and the run's status, metrics and per-frame trace.

Design notes

- **Evidence chain.** An alert links to events (and optionally an entity); an event links to observations; an observation is one frame. Any claim can be walked back to frame identifiers, which is what the analyst cites.
- **Timestamps** are stored twice: the exact ISO-8601 text with its UTC offset (so the original timezone round-trips) and a fixed-width UTC string used for ordering and range filters, because comparing ISO strings with different offsets would order them wrongly.
- **Attributes** are open-ended (whatever a model reports), so they are stored as canonical JSON and searched through FTS5; they are re-validated by Pydantic models when read back.
- **Search safety.** Search text is never passed to FTS5 as-is: each word is extracted and quoted and the words are ANDed, so input such as `'; DROP TABLE x; --` is treated as plain words. Results are ranked with BM25 and can be combined with time, zone and object filters. FTS5 has no synonyms (a search for *truck* does not find *pickup*) or typo tolerance.
- **Integrity.** Foreign keys are enforced; ingestion is one atomic transaction (a duplicate ingest is rejected and changes nothing); `build-events` and `analyze` replace their derived data in one transaction, so they are idempotent. Schema changes are additive and versioned with `PRAGMA user_version`.

Why SQLite. One file, no infrastructure, transactional, and it supports structured filters (time, zone, object type) and full-text search in the same database, so a frame, its objects and its search entry are written in one transaction. It is enough for the assignment's indexing requirement ("show all truck events"-style queries by time, zone, object or keyword); embeddings and vector search were deliberately left out of scope.

---

5. Vision and video pipeline

5.1 Video source, sampling and the frame gate

- **VideoSource** validates the file when it is opened: it exists and is non-empty, OpenCV opens it, the reported FPS is finite in (0, 1000], the frame count is positive, and the first frame decodes. Its identity is the SHA-256 of the file bytes.
- **Sampling** is deterministic and time-based: sample *k* is taken at frame index `floor(k × every × fps + 0.5)` (default every 2 s), so the same video and settings always select the same frames, including for fractional FPS. Frames are decoded sequentially, which is simple and codec-safe.
- **Frame gate.** The mean absolute difference between 64×36 greyscale copies of a frame and the *last accepted* frame is compared with a threshold (default 6.0, uncalibrated). The first frame is always accepted, and the reference resets on a zone change. The gate is a pixel test: it is not motion detection, tracking or semantic analysis.
- **Carried observations.** By default a skipped frame is represented by a copy of the last accepted observation, labelled `<provider>+carried`, so a static scene stays one continuous activity for the aggregator. *A carried observation is temporal propagation of an earlier answer, not a new model inference.* `--skipped drop` stores nothing for skipped frames instead.

5.2 Provider abstraction and Gemini

- **Provider contract.** A real model returns a strict `ProviderOutput` (description; objects with type, activity, coarse attributes and optional confidence). Unknown keys, wrong types and bad ranges are errors. The pipeline, not the model, assigns zone, time and identifiers.
- **Gemini FrameDescriber** (official `google-genai` SDK). The frame is downscaled only if its longest side exceeds 1024 px, JPEG-encoded in memory (quality 85) and sent with the shared instructions, which tell the model to describe only visible objects and not to judge suspicion, intent, identity or severity. Native structured output is requested, then the reply is validated again by the project's own strict parser; there is no regex extraction or silent repair.

- **Retry and failure policy.** Missing key, authentication failure, unknown model and rejected requests fail immediately (no fallback model). Rate-limit, timeout, network and server errors get at most two retries (2 s and 5 s); malformed output gets exactly one retry. The request timeout is 60 s. Error text is scrubbed of the key and the raw SDK exception is not chained. The model comes from `--model`, `GEMINI_MODEL` or a code default.
- **Confidence.** Gemini reports no calibrated probability, so none is requested. Objects are stored with a documented uncalibrated placeholder (0.5); it is not a probability and nothing downstream uses it for decisions.
- **Cassettes.** Recorded responses are keyed by `<video sha256[:16]>:<frame index>` and hold descriptions only (no images, keys or absolute paths). Modes: *replay* (no key, no API call; a missing frame is an error), *record* (calls the model only for frames not yet recorded, writes the file even if a later call fails, so a rerun resumes) and *live*. A cassette recorded for another video, or by another model, is refused.
- **API key handling.** `GEMINI_API_KEY` is read from the environment or a local, gitignored `.env`; it is never printed, logged or stored in the database or a cassette.

> **No accuracy claim.** Frame descriptions from Gemini are used as given; how accurate they are was not measured, because no annotated ground truth exists for the real clip (Section 10). The recorded Gemini runs show that the integration works, not that the descriptions are correct.

---

6. Event and entity reasoning

6.1 Temporal event aggregation

The aggregator is a streaming algorithm over chronological observations. It is *temporal association, not object tracking*: it never looks at pixels.

- Detections of the same object type in the same zone join an open event when their stable attributes are compatible (person: `clothing`; vehicle: `color`, `body_type`; a value missing on either side is treated as unknown). A detection with incompatible attributes starts a separate event, so two people standing together stay two events.
- An event closes on a zone change, when the next matching detection is more than 180 s later, or after more than 2 consecutive frames in its zone without a matching detection (*missed-frame tolerance*: the drone looked, the detector missed). Otherwise it extends, keeping first seen, last seen, duration and ordered evidence.
- One detection can join at most one event per frame; ties go to the most recently seen event. Event identifiers (`EVT-P-001`, `EVT-V-001`, …) follow first-seen order and are deterministic.
- Attributes are compared after normalisation (for example “dark blue” and “blue” agree, “grey” becomes “gray”); the stored attributes keep their original text.

6.2 Coarse vehicle entity linking

- Only *vehicle* events are linked, into entities such as `ENT-V-001`. Two events join when their normalised colour and body type are equal, they do not overlap in time, and the required attributes are present; missing evidence prevents a link and nothing is guessed. An explicit shared plate confirms an entity; conflicting plates never link, even with identical coarse attributes.
- Entities are labelled “similar blue pickup”. A coarse match is not physical identity; only a match level of `confirmed` (explicit equal plate evidence) says more, and no plate reading is performed.
- **People are never identity-linked.** Separate person events stay separate, and the analyst may not say that the same person appears in separate events. There is no face recognition and no appearance model.

Known limitations (measured in Sections 9 and 10): coarse matching can group different vehicles that share colour and body type; a vehicle described with inconsistent colours splits into two entities; free-text attributes from a real model fragment events; and the event model has no positions, so it cannot say which of several same-type objects in a frame is which.

---

7. Security detection rules

Rules are deterministic functions over stored events, entities and telemetry, configured in `config/security_rules.yaml`. They decide whether an alert exists; no LLM takes part. Each alert carries a reason, the numbers the rule compared, and links to the triggering events, the entity (if any) and the frames.

The eight rules (default configuration)

Rule	Fires when	Severity

Rule	Fires when	Severity
Loitering	a person event in `main_gate` lasts at least 120 s (inclusive); the same alert escalates to critical at 300 s	medium, then critical
After-hours presence	a person event overlaps the closed window 22:00-06:00 (start inside, end outside; the window crosses midnight); time of day uses the event's own UTC offset unless a property offset is configured	medium
Repeat vehicle	a coarse vehicle entity has at least 3 separate appearances; the wording is "a *similar* ... was observed in N separate appearances" and says a coarse match does not confirm the same vehicle	low
Restricted-zone entry	a person or vehicle event occurs in `restricted_zone`	high
Low battery (telemetry)	battery at or below 20%; critical at or below 10%; resolves once back at 25% or more	medium, then critical
Signal loss (telemetry)	signal below 40% is degraded, below 15% is lost, and a gap of more than 30 s in delivered telemetry counts as a lost link	medium, then high
Altitude anomaly (telemetry)	altitude outside 5-120 m; beyond 2 m or 150 m it is high	medium, then high
Geofence violation (telemetry)	the drone is more than 100 m from the property centre; critical beyond 150 m	high, then critical

- **Identifiers and duplicates.** Alert ids are deterministic, `ALR-<RULE>-<subject or episode>`, and an identical (rule, subject or episode) pair yields exactly one alert; re-running on unchanged data gives the same alerts.
- **Severity ladders and lifecycle.** A rule may raise an alert at one severity and escalate the *same* alert when the condition worsens (loitering: warning at 120 s, critical at 300 s); the alert id never changes. Each alert has a status history (open, escalated, acknowledged, resolved), a confidence, a verdict and a recommended action written for the rule and severity. State alerts (loitering, safety) resolve when the condition ends; incident alerts (after hours, repeat vehicle, restricted zone) stay open until an operator acts, and only a rule or an operator changes a status, never a language model. A loitering alert also resolves when its event closes because the camera moved to another zone, which is a simplification: the last frame in the zone still showed the person. Different rules may alert on the same event because they describe different risks (a night-time loiterer yields a loitering and an after-hours alert).
- **Evidence.** `alerts --show <id>` prints the reason, the rule facts and the evidence chain down to frames (or, for drone-safety alerts, telemetry samples).
- **Progressive processing.** `run --scenario NAME` drives the same rules one tick at a time. Alerts are stored and printed the moment they fire, and with sampling switched off the final database equals the batch pipeline's (a test asserts it for the two main scenarios). Only scripted scenarios run progressively; a real video is ingested first and turned into events and alerts by `build-events` and `analyze`.
- **Documented false positives and negatives** include a delivery driver waiting at the gate (loitering), authorised night staff (there is no allow-list), a person missed for longer than the tolerance splitting into events that stay under the threshold, and thresholds that were hand-set on scripted data rather than validated on real footage.

8. Analyst agent

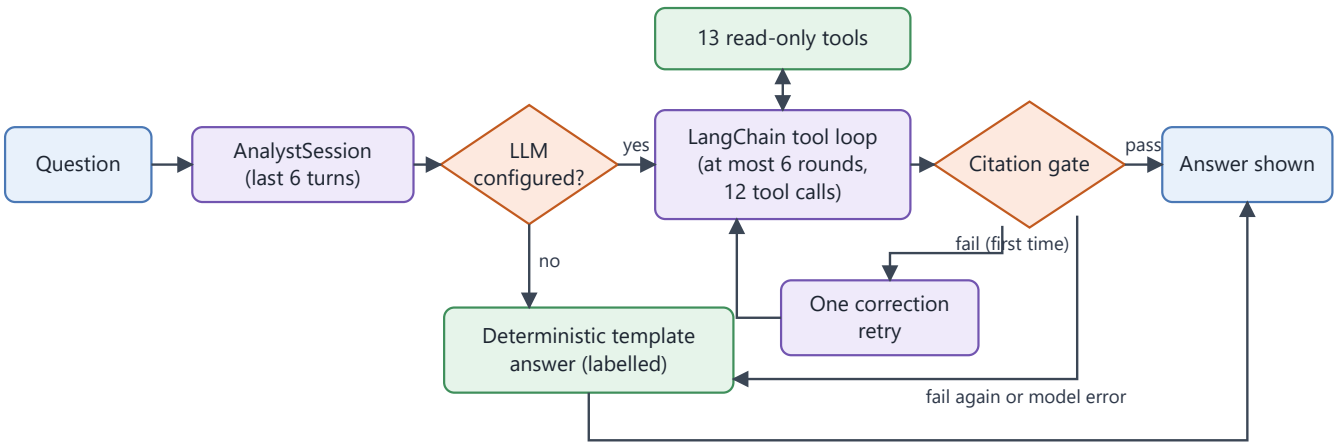


Figure 3. Analyst path. The template analyst is both the offline mode and the fallback; nothing is shown that has not passed the citation gate or come from the deterministic template.

8.1 Tools and the tool loop

- The analyst reaches the database only through thirteen read-only, bounded tools: `query_events`, `search_events`, `search_frames`, `get_event`, `get_frame`, `get_alert`, `get_evidence`, `entity_history`, `list_entities`, `list_alerts`, `telemetry_at`, `get_timeline`, `summarize_run`. There is no SQL or generic execution tool; invalid arguments return an error object instead of raising.
- Every result has an explicit bound (default 8 and at most 15 observations per search; 20 events, alerts or entities; 15 evidence observations per event; descriptions cut to 200 characters) and reports `total`, `returned` and `truncated`.
- With an LLM configured, a LangChain chat model (Gemini) runs a hand-written tool-calling loop bounded to 6 model rounds and 12 tool calls per answer. LangChain provides only the model and tool binding; there is no agent framework. The analyst model is configured separately from the vision model (`ANALYST_MODEL`).
- The system prompt requires: facts only from tools, inline citations, observation kept apart from inference, "similar" vehicle wording, no claim that the same person appears in separate events, no identity or intent claims, alerts reported exactly as stored, and "Insufficient evidence" when the evidence does not support an answer.

8.2 Citations and the citation gate

Answers cite internal references with tokens such as `[event:EVT-V-001]`, `[alert:ALR-REPEAT-ENT-V-001]`, `[entity:ENT-V-001]`, `[obs:obs-10]` and `[frame:10]`. An LLM answer is shown only if all of the following hold:

1. every cited id exists in the database and was returned by a tool in this conversation;
2. the answer cites something, unless it says "Insufficient evidence";
3. it contains no malformed token and no citation-shaped token of an unknown type (for example `[evt:...]`);
4. it contains no identity or intent wording from a short phrase list ("the same person", "burglar", "criminal", ...), unless a negation cue stands in the same clause within eight words before it.

On failure the model gets one correction turn naming the problems; if the second answer also fails, or the model call fails (authentication, quota, network), the deterministic template answer is returned instead, labelled as such. Each LLM-backed answer reports call metrics outside the answer text (model, rounds, tool calls, correction attempts, fallback yes/no, latency).

> **What the gate is not.** It checks form, existence and retrieval. It does *not* establish that the sentence around a citation is true, and it is not semantic entailment. Section 9.7 measures the resulting gap.

8.3 Offline template analyst, sessions and truncation

- The **template analyst** is deterministic and needs no key. It pattern-matches a limited set of question shapes (alerts by rule or id, events, entities, zone summaries, blue-pickup-style vehicle questions, after-hours people, objects, run summary, a labelled keyword search) and otherwise answers "Insufficient evidence" and lists what it can answer. Output is labelled "template answer (deterministic, no LLM)".
- **Sessions.** A session keeps the last six question-answer pairs in memory (no persistence) and passes them to the LLM. Template follow-ups such as "Where was it seen?" resolve a pronoun only when the previous answer named exactly one entity, alert or event; otherwise the answer is "Insufficient evidence" rather than a guess. `ask --follow-up` and `--interactive` drive a session from the command line.
- **Summaries.** `summarize` renders bounded structured facts (counts, time span, zones, object counts, events with their alerts, entities, and data-driven limitations); with an LLM the same facts may be rewritten as prose under the same gate. A follow-up such as "What objects were observed?" is answered from stored evidence, never by reprocessing the video.
- **Truncation transparency.** Running the analyst on the real-video database showed that the template analyst silently dropped results (for example 3 of 11 vehicle entities and 20 of 25 events). It now states what was cut ("showing 20 of 25 events, earliest first; 5 not shown"), lists the remaining entities briefly, answers general people-and-vehicles questions for both categories, and adds nothing when nothing was cut; answers that were not truncated were verified byte-for-byte unchanged.

---

9. Experimental evaluation

9.0 Method and principles

The evaluation harness (`python -m drone_agent evaluate`) is offline and deterministic; two full runs produce identical functional results, and every table below was generated by executable code and cross-checked against a fresh run when this report was built. Principles:

- **Baseline is the production default.** Ablations change one setting inside the harness (for example `entities=[]`, a different `max_missed_frames`, gate threshold 0); production code is untouched.
- **No tuning against the fixtures.** Thresholds were not adjusted to make results look better; the gate thresholds swept were chosen from the observed score distribution, and the default stayed 6.0. Negative results were kept.
- **Scripted detections do not measure perception.** Fixtures are authored or generated, so results say whether the code behaves as designed, not how well a vision model sees.
- **Kind of evidence is labelled** on every result. All results in Section 9 are Controlled / synthetic unless stated otherwise; they must not be read as real-world accuracy.

9.1 Frame-gate ablation Controlled / synthetic

Question. How many vision-model calls does the gate avoid, and which meaningful changes does it miss? **Method.** 40 generated 320×180 frames with seeded noise and exactly known changes: four meaningful object changes (a large object appearing and leaving, a medium object, a small object) and two illumination-only steps (+8 and +25). The real gate was run at eight thresholds; a change counts as detected when the frame at which it happens is accepted. No model is called.

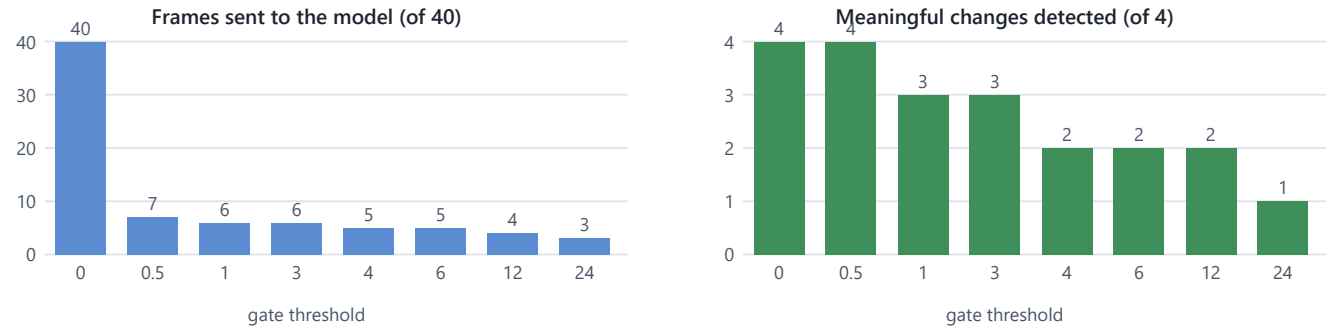


Figure 4. Effect of the gate threshold on the synthetic fixture. The production default is 6.0.

Frame-gate threshold sweep (controlled synthetic fixture: 40 candidate frames, 4 meaningful and 2 illumination-only changes)

threshold	accepted	skipped	call_reduction_pct	meaningful_detected	meaningful_missed	meaningful_recall	irrelevant_accepted
0.0	40	0	0.0	4	0	1.0	2
0.5	7	33	82.5	4	0	1.0	2
1.0	6	34	85.0	3	1	0.75	2
3.0	6	34	85.0	3	1	0.75	2
4.0	5	35	87.5	2	2	0.5	2
6.0	5	35	87.5	2	2	0.5	2
12.0	4	36	90.0	2	2	0.5	1
24.0	3	37	92.5	1	3	0.25	1

Gate score of each known change (a static frame scores at most 0.388)

change	meaningful	score vs previous frame
large object appears	yes	19.622
illumination step +25	no (illumination only)	19.908
small object appears	yes	0.952
medium object appears	yes	3.291
illumination step +8	no (illumination only)	6.196
large object leaves	yes	26.538

Measured. At the default threshold 6.0 the gate sent 5 of 40 frames (87.5% fewer calls) and detected 2 of 4 meaningful changes; it accepted 2 of 2 illumination-only changes. A static frame scores at most 0.388; the small object scores 0.952 and the medium object 3.291, while an irrelevant +8 illumination step scores 6.196. Threshold 0.5 sent 7 frames (82.5% fewer) and detected 4 of 4 on this fixture.

Reading. Pixel change and semantic importance are not equivalent: a small but meaningful change scored below an irrelevant illumination change. A missed change is silently absorbed because the reference frame is not updated. The result on this clean

synthetic fixture is not evidence that 0.5 is better in general (real footage has noise, shadows and camera motion), and the default was intentionally not changed. **Limits.** One 40-frame sequence with a smooth background and clean rectangles.

9.2 Temporal compression Controlled / synthetic

Method. The production aggregator over the `security_events` scenario, the `normal_day` scenario and a controlled 60-frame dwell.

Observations to events (controlled scripted and generated fixtures)

fixture	observations	detected_objects	events	compression_ratio
security_events	27	20	7	2.857
normal_day	11	11	5	2.2
long_dwell_controlled	60	90	2	45.0

Measured. On `security_events`, 20 detections from 27 observations became 7 events (ratio 2.86); every detection remains linked to an event as evidence and every event span equals the span of its evidence. In the dwell fixture, 90 repeated detections became 2 events. **Reading.** Repeated frame-level detections become bounded temporal units; the ratio describes merging, not correctness (a wrong merge also raises it).

9.3 Flicker tolerance Controlled / synthetic

Question. Does `max_missed_frames` stop one undetected frame from splitting a continuous event? **Method.** The loitering episode in `security_events` (frames 1-9, frame 5 has no detection) with tolerance 0 versus the default 2, everything else identical, plus a gap sweep.

Measured. Default (2): 1 event of 480.0 s and 1 loitering alert. Tolerance 0: 2 events (180 + 180 s) and 2 loitering alerts.

Gap sweep: one person seen for 4 frames, a gap of N undetected frames, then 4 more frames (controlled fixture)

fixture	max_missed_frames	person_events	durations_s	loitering_alerts
gap=0	0	1	[420.0]	1
gap=0	2	1	[420.0]	1
gap=1	0	2	[180.0, 180.0]	2
gap=1	2	1	[480.0]	1
gap=2	0	2	[180.0, 180.0]	2
gap=2	2	1	[540.0]	1
gap=3	0	2	[180.0, 180.0]	2
gap=3	2	2	[180.0, 180.0]	2
gap=4	0	2	[180.0, 180.0]	2
gap=4	2	2	[180.0, 180.0]	2

Reading. Loitering is decided from an event's duration, so one undetected frame can remove an alert. In the sweep the default tolerance keeps one event for gaps of up to 2 frames and splits at 3; the tolerance is bounded on purpose. **Limits.** Misses are inserted by hand at 60 s spacing; real detector misses are correlated and longer.

9.4 Entity-memory ablation and negative controls Controlled / synthetic

Method. The events of `security_events` with coarse entity linking versus every vehicle event independent (done in the harness), plus vehicle-linking controls checked against the linker's documented contract.

security_events with and without coarse entity linking (controlled scripted scenario)

setting	vehicle_events	vehicle_entities	multi_appearance_entities	repeat_vehicle_alerts
linking on	4	2	1	1
linking off	4	0	0	0

Vehicle-linking controls (controlled synthetic sightings)

control	linked	documented contract respected	false link
different colour	False	True	False

control	linked	documented contract respected	false link
different body type	False	True	False
missing body type	False	True	False
conflicting explicit plates	False	True	False
two real pickups, same coarse attributes (false-link limitation)	True	True	True
same plate, different colour (explicit plate evidence)	True	True	False
overlapping in time (cannot be one vehicle)	False	True	False

Measured. With linking, the blue, dark-blue and blue pickups in three zones form one entity and produce 1 repeat-vehicle alert; without linking there are 0 entities and 0 alerts. The documented contract was respected in 7 of 7 controls. **Reading.** Linking is what makes the repeat rule and "seen again" questions possible. The controls also show its limit: two genuinely different pickups with identical coarse attributes are linked (a false link), which is why the wording is always "similar vehicle". This is attribute-only grouping, not re-identification or tracking.

9.5 Rule evaluation Controlled / synthetic

Method. 22 authored scenarios with expected alert counts per rule, run through the production aggregator, linker and rule engine. Each (scenario, rule) pair is one cell: matched alerts are true positives, surplus alerts false positives, missing alerts false negatives, and an empty-versus-empty cell is one true negative. A zero denominator is reported as N/A. `security_events` and `normal_day` are reported separately as development fixtures.

Rule agreement with hand-written labels (controlled synthetic scenarios only)

rule	tp	fp	tn	fn	precision	recall
loitering	4	0	18	0	1.0	1.0
after_hours_presence	5	0	17	0	1.0	1.0
repeat_vehicle	1	0	21	0	1.0	1.0
restricted_zone_entry	3	0	19	0	1.0	1.0

Measured. All 22 scenarios matched their labels. **Reading.** This confirms the implemented boundaries (inclusive 300 s threshold, closed-window start inclusive and end exclusive, midnight crossing, missing-evidence non-link) without an LLM. **Limits.** The labels and the rules were written by the same author, the detections are scripted, and these are agreement figures on synthetic scenarios, *not real-world precision or recall*.

9.6 Citation-gate adversarial suite Controlled / synthetic

Method. 29 authored analyst outputs across ten categories were run through the production analyst session (a scripted stand-in for the model; tool calls performed first so citations can be "retrieved"). An output counts as accepted only if it is shown as an LLM answer on the first attempt.

Citation-gate decisions on the authored adversarial suite

category	cases	expected	correct decisions
valid_retrieved	3	accept	3
nonexistent	5	reject	5
real_not_retrieved	3	reject	3
malformed	4	accept	4
no_citation	3	reject	3
insufficient_evidence	1	accept	1
identity_claim	3	reject	3
intent_claim	3	reject	3
negated_identity	2	accept	2
uncertainty_vehicle	2	accept	2

Measured. 29 of 29 decisions were correct (19 outputs expected to be rejected, 10 expected to be accepted). **Important qualification.** The first run of this suite made two wrong decisions: a citation-shaped token of an unknown type such as `[evt:...]` was silently ignored (a false accept), and a negation more than two words before "the same individual" was not

recognised (a false reject). Both validator behaviours were then fixed against the same cases and the fixtures were left unchanged. Because the validator was corrected against this suite, 29/29 is a result on the authored fixture set and *is not independent evidence of general citation reliability*.

9.7 Semantic-faithfulness probe Controlled / synthetic

Question. Which unsupported claims still pass the gate when every citation is valid? **Method.** 22 authored answers whose citations all exist, were retrieved and are well formed, but whose claims are false or overreaching (wrong zone, duration, severity, rule, appearance count, type, activity or start time; a persistent-tracking claim; identity wording outside the phrase list; intent inference; a claim that the analyst raised an alert), plus 5 true control answers. An evaluation-only prototype then checks *structured* claims (event zone, type, activity, duration, start time; alert severity and rule; entity appearance count) by exact comparison with the stored evidence.

Citation-valid answers, by category (authored from the evidence model)

category	answers	claims true	accepted by citation gate	accepted by gate + structured checker
control_true	5	yes	5	5
wrong_zone	2	no	2	0
wrong_duration	2	no	2	0
wrong_severity	2	no	2	0
wrong_rule	1	no	1	0
wrong_count	2	no	2	0
wrong_type	1	no	1	0
wrong_activity	2	no	2	0
wrong_time	1	no	1	0
capability_overclaim	2	no	2	2
identity_evasion	4	no	4	4
intent_evasion	2	no	2	2
boundary_overclaim	1	no	1	1

Measured. The citation gate accepted 22 of 22 unsupported answers, and 5 of 5 true controls, so it cannot tell them apart. Of the 13 unsupported answers that could be expressed as structured claims, the prototype checker caught 13; 9 unsupported answers still pass gate plus checker, all in classes the checker does not cover (capability overclaims such as “tracking”, identity wording, intent inference, and a claim that the analyst raised an alert). **Reading.** This characterises the boundary of what citation validation guarantees. The checker verifies claims that were already structured; it does not read prose, its catch rate on its own claim classes is true by construction, it lives in the evaluation layer only, and it is *not* general semantic entailment. A live analyst answer also used the phrase “vehicle tracking” (the system does coarse attribute grouping, not tracking) and passed the gate.

9.8 Ablation summary Controlled / synthetic

Ablation summary (every value computed by the harness)

component	metric	baseline_setting	baseline	ablated_setting	ablated	effect
Frame gate	vision-model calls per 40 candidate frames (proxy)	gate threshold 6 (default)	5	gate off (threshold 0)	40	-35 calls (87.5% fewer)
Frame gate	meaningful scene transitions detected	gate threshold 6 (default)	2/4	gate off (threshold 0)	4/4	-2 transitions
Temporal aggregation	evidence units (security_events)	events (aggregation on)	7	raw frame-level detections	20	-13 units (ratio 2.857)
Flicker tolerance	events from one loitering episode	max_missed_frames=2 (default)	1	max_missed_frames=0	2	+1
Flicker tolerance	loitering alerts from that episode	max_missed_frames=2 (default)	1	max_missed_frames=0	2	+1
Flicker tolerance	highest loitering severity from that episode	max_missed_frames=2 (default)	critical	max_missed_frames=0	medium	critical -> medium
Adaptive sampling	meaningful changes detected within 5 frames (generated streams)	adaptive (pixel + telemetry + event watch + refresh)	0.7556	pixel gate only (previous default)	0.4444	-0.311

component	metric	baseline_setting	baseline	ablated_setting	ablated	effect
Adaptive sampling	model calls per 600-frame stream	adaptive	36.5	pixel gate only (previous default)	11.375	-25.1 calls
Adaptive sampling	model calls per detected change	adaptive	4.29	pixel gate + refresh every 15 frames	5.69	+1.40
Streaming alerts with ground truth	alert F1 (perfect scripted perception)	10 scenarios, 18 expected alerts	1.0	n/a	n/a	latency median 0.0 s, max 10.0 s
Entity memory	repeat-vehicle alerts (security_events)	coarse linking on (default)	1	each vehicle event independent	0	-1
Entity memory	multi-appearance vehicle entities	coarse linking on (default)	1	each vehicle event independent	0	-1
Citation validation	invalid outputs accepted (of 19 adversarial)	citation gate on	0	no gate	19	-19
Claim-consistency prototype (eval-only)	semantically unsupported answers accepted (of 22)	citation gate + structured claim checker	9	citation gate alone (production)	22	-13

9.9 Streaming ground truth Controlled / synthetic

Question. Does a progressive run reproduce the events, alerts, severity ladders and timing that each scenario's author wrote down beforehand? **Method.** 10 scenario files carry a machine-readable `expect:` block written before the system was run against them (camera scenarios such as a midnight loiterer, an intruder walking to the garage, a repeat pickup, a benign delivery and a flickering detector, and emergency scenarios for low battery, signal loss and a geofence violation). Each is run through the streaming engine with the default rules and the adaptive sampler on and matched one to one with its labels. F1 is 2TP / (2TP + FP + FN), so a scenario whose every prediction is wrong scores 0, not N/A.

Streaming run against author-written labels (controlled synthetic scenarios only)

scenario	expected_alerts	tp	fp	fn	f1	forbidden_hits	events	exact
benign_delivery	0	0	0	0	N/A	0	3/3	True
flicker_noise	1	1	0	0	1.0	0	1/1	True
geofence_violation	1	1	0	0	1.0	0	0/0	True
intruder_garage	2	2	0	0	1.0	0	2/2	True
loiter_midnight	2	2	0	0	1.0	0	1/1	True
low_battery	1	1	0	0	1.0	0	0/0	True
normal_day	0	0	0	0	N/A	0	1/1	True
repeat_pickup	1	1	0	0	1.0	0	4/4	True
security_events	7	7	0	0	1.0	0	7/7	True
signal_loss	3	3	0	0	1.0	0	0/0	True

Measured. 10 of 10 scenarios matched exactly, with 18 true positives, no false positives and no false negatives over all expected alerts; the severity ladder, rung timing and final lifecycle status were all right. Detection latency is bounded by the frame spacing (60 s) for camera alerts and the 10 s telemetry tick for safety alerts. **Limits.** Labels and rules share an author and detections are perfect, so this shows the implementation matches its author's intent, not real-world accuracy.

9.10 Adaptive sampling Controlled / synthetic

Question. Does combining pixel change, telemetry change, event state and a staleness bound detect more meaningful changes per model call than a pixel gate? **Method.** Eight seeded 600-frame generated streams with small, medium and large changes and illumination steps, using the real gate score and the real sampler. The saving is reported together with the changes missed.

Model calls against changes detected (8 generated 600-frame streams)

policy	calls_mean	avoided_pct_mean	recall	calls_per_detected	missed_small
every frame (gate off)	600.0	0.0	1.0	53.33	0
pixel gate, threshold 6 (previous default)	11.375	98.088	0.4444	2.27	31
pixel gate, threshold 3	12.125	97.963	0.5111	2.11	31
pixel gate + refresh every 15 frames	45.5	92.425	0.7111	5.69	13

policy	calls_mean	avoided_pct_mean	recall	calls_per_detected	missed_small
pixel gate + refresh every 30 frames	26.125	95.65	0.5778	4.02	23
adaptive (pixel + telemetry + event watch + refresh)	36.5	93.912	0.7556	4.29	16
adaptive without telemetry signal	31.75	94.7	0.6111	4.62	22
adaptive without event watch	17.625	97.075	0.7	2.24	19
adaptive without forced refresh	32.5	94.588	0.7333	3.94	18

Reading. The previous pixel-only default avoided the most calls and missed the most meaningful changes; adaptive sampling bought recall with calls, and the fair comparison is a pixel gate with a fixed refresh. For scripted frames, which have no pixels, the sampler's scene signal is a multiset of object class, activity and attributes, so an activity-only change or an extra identical-looking object counts as a change. **Limits.** Synthetic streams and an assumed 30% telemetry-cue rate; the adaptive sampler is used by the scripted runtime, while real video still uses the pixel gate alone.

9.11 Scalability

Generated-observation timings at 100, 1,000, 10,000 and 50,000 observations are in Section 11 (Measured on this machine); a later benchmark of the real pipeline and the analyst tools at 10,000 and 100,000 observations is committed under `bench/`.

10. Real-video experiment

Real video

> **Purpose and scope.** One real clip was run through the whole pipeline with real Gemini output. It closes the gap that scripted detections do not exercise a real vision model. **The clip is a high-angle traffic-intersection video, not a controlled fixed-property security benchmark**, and there is no perception ground truth. It validates end-to-end integration; it does not measure perception accuracy or security-detection performance.

Input and processing

Item	Value
File	`output/real/clip.mp4` (local, never committed), 10,797,538 bytes; provenance and licence are not recorded in the repository
Video	H.264 (High) in MP4, 1920×1080, 25 fps constant, 617 frames, 24.68 s, no audio
Sampling	every 3 s: 9 candidates at offsets 0, 3, …, 24 s (frame indexes 0, 75, …, 600); all decoded
Gate	threshold 6.0 (unchanged default): **8 accepted, 1 skipped**; the skipped sample (21 s, score 5.36) was carried forward from frame 450
Gemini	`gemini-3.6-flash`; **8 API calls, 0 retries, 76.1 s total API latency** (about 9.5 s per call), as printed in the record run's summary (latency and retries are not stored in the cassette or database)
Cassette	8 validated responses (descriptions and objects only); keyed by video hash prefix `d7e3ceeba55902be` and frame index
Telemetry	**Simulated**: 3 samples; start `2026-09-29T14:00:00+05:30` and zone `main_gate` supplied on the command line and applied to the whole clip
Confidence	uncalibrated placeholder 0.5 for every object

Results of the deterministic stages on the real observations

Stage	Result
Observations / objects	9 observations (8 Gemini, 1 carried); 33 detected objects (17 person, 16 vehicle; 3 of them belong to the carried observation, so the 8 Gemini responses contained 30)
Events	25 (13 person, 12 vehicle); compression ratio 1.32 (the scripted `security_events` gives 2.86); only 5 of 25 events span more than one frame, 20 last 0 s
Entities	11 coarse "similar" vehicle groups; 10 have one appearance and 1 (a similar white van) has two
Alerts	**0**

10.1 Why there are zero alerts

None of the four camera rules could fire on this input (the run was recorded before the telemetry-safety rules existed, and its telemetry is simulated), for reasons unrelated to what the footage shows: loitering needs an event of at least 300 s in `main_gate` (the clip lasts 24.68 s and the longest event is 15 s); after-hours presence does not apply because the simulated start is 14:00 and closed hours are 22:00-06:00; the simulated zone is not restricted; and repeat vehicle needs three separate appearances (the

maximum here is two). Zero alerts is **not** a security-detection result, not a specificity measurement and not evidence that nothing noteworthy happened. Any alert on this clip would equally be a consequence of the simulated time and zone, not of the footage.

10.2 Findings

1. **Event fragmentation from free-text attributes.** Person events require the `clothing` text to match after lower-casing. Gemini's wording varies ("dark clothing", "dark coat", "dark coat, jeans", "dark jacket and jeans", ...), so most detections start their own event. A frame can hold up to three people and three vehicles and the observations carry no position, so which object is which cannot be inferred; an event unmatched for 3 consecutive sampled frames (9 s) closes. Result: 20 of 25 events are single-frame.
2. **Inconsistent VLM attributes.** For example, frame 300 lists a white hatchback and a blue vehicle with no body type, frame 375 a blue hatchback and frame 600 a black hatchback; colour and body type are not stable descriptions of the same vehicle across samples.
3. **Coarse vehicle-linking limitation.** The one multi-appearance entity links a "white van" at frame 0 with a white van at frames 375-525, 15 s apart; the frame-375 description calls it "a white ambulance" while its attribute says `van`. Whether it is one vehicle cannot be determined from this data. The system labels it a coarse, unconfirmed match; the case illustrates the false-link risk measured in Section 9.4.
4. **Carried observation.** Frame 525's objects are a copy of frame 450's; three events extend over frame 525, so their durations (6 s, 3 s, 3 s) include propagation, not a new detection.
5. **No ground truth.** Gemini's descriptions were not compared with a human annotation.

10.3 Replay verification and analyst findings

Replaying the cassette (`--provider cassette`, no key, no API call) reproduced the run: 8 cassette replays, 9 observations and 33 objects, with observations and detected objects identical to the recorded run except the `source` label, which reads `cassette` instead of `gemini:gemini-3.6-flash`. The recorded artifacts (video, cassette, databases) stay under the gitignored `output/real/` and are not part of the repository.

Asking the offline analyst about this run exposed the truncation described in Section 8.3. Before the fix, "What people and vehicles were observed?" returned 3 of 11 vehicle entities and no people, and the main-gate and objects answers listed 20 of 25 events with no note. After the fix they state "showing 20 of 25 events, earliest first; 5 not shown", give full history for 3 of the 11 entities and list the other 8 briefly, and report both people and vehicles.

11. Scalability

Measured on this machine

Method. Deterministic generated observations (empty stretches, people and vehicles that dwell for a few frames in a zone) at four sizes. Each in-memory stage was timed five times and SQLite queries twenty times; the median is reported. SQLite ingest and derived-table writes are single runs. No network or API call is involved, and the vision model's latency, which dominates a real deployment, is not part of this benchmark.

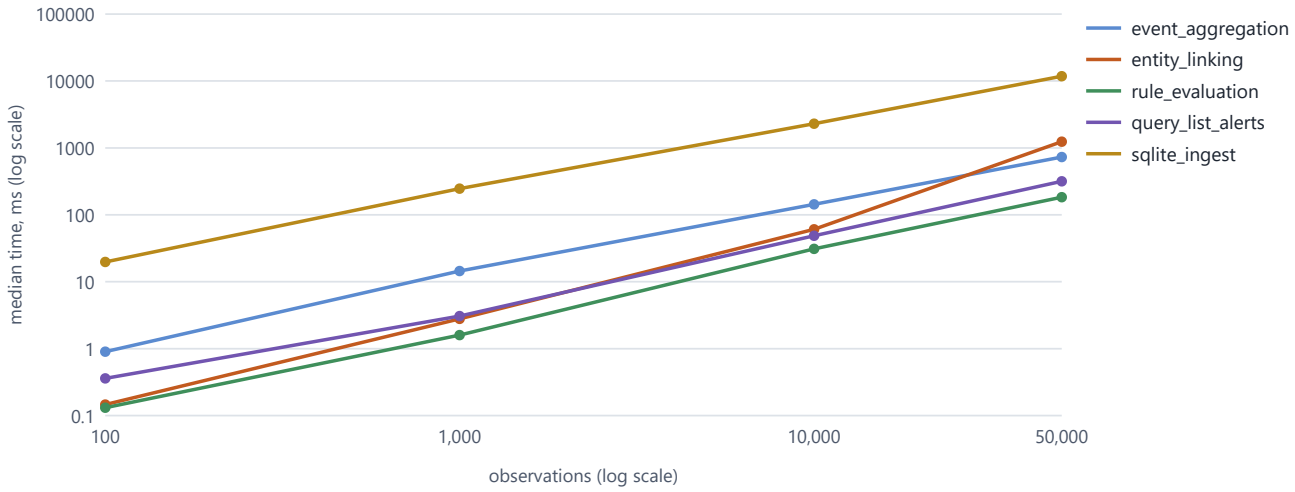


Figure 5. Median time by number of generated observations (log-log; a straight line of slope 1 would be linear growth).

Median time in milliseconds by number of generated observations (SQLite ingest and derived-table writes are single runs)

stage	100	1,000	10,000	50,000
event_aggregation	0.901	14.483	143.559	732.383
entity_linking	0.146	2.788	60.701	1242.49
rule_evaluation	0.131	1.598	31.04	183.465
query_fts_keyword_top10	0.097	0.724	1.254	4.144
query_filter_zone_type_window	0.818	1.551	2.108	1.538
query_count_matching_zone	0.088	0.413	2.572	22.169
query_list_events_by_zone	0.282	1.731	15.832	110.172
query_list_alerts	0.359	3.075	48.709	317.248
sqlite_ingest	19.782	246.523	2298.01	11790.6
sqlite_persist_derived	15.743	36.001	253.199	914.648

Measured. Event aggregation took a median 0.901 ms for 100 observations and 732.383 ms for 50,000; rule evaluation 183.465 ms and the keyword search (top 10) 4.144 ms at 50,000. The empirical scaling exponent between the two largest sizes (time ~ size^k, k=1 is linear) was 1.01 for event aggregation, 1.10 for rule evaluation and **1.88 for entity linking**.

The entity-linking bottleneck. Entity linking is clearly superlinear at the largest sizes: 1242.49 ms at 50,000 observations, longer than event aggregation. Reading the code, the linker recomputes each candidate group's last-seen time (a maximum over the group's events) for every comparison, which would explain the growth. This is an observation from the code, not a tested cause, and it was *not* fixed: the behaviour was preserved so the measurement stays a baseline.

Measurement environment (this machine only)

property	value
python	3.11.9
platform	Windows-10-10.0.22631-SP0
processor	Intel64 Family 6 Model 170 Stepping 4, GenuineIntel
cpu_count	14
sqlite_version	3.45.1

Limits. One machine and one session, synthetic observations with a small attribute vocabulary, in-memory stages that exclude reading rows back from SQLite. The results are measurements on this machine and make no claim of production-scale readiness, of behaviour on other hardware or with concurrent access.

12. Testing Automated tests

Current result. 963 tests are collected and pass, all offline, in roughly 30 to 90 seconds on the development machine.

Test organisation (42 test modules)

Area	Modules	What is verified
Domain and simulation	models, clock, telemetry simulator, sync, scenario, simulate	strict immutable models, timezone-aware time, seeded determinism, midnight crossing, dropouts and the stale-telemetry boundary, scenario validation
Storage and indexing	store, ingest	schema and foreign keys, exact round trips, timezone-safe ordering, filters, FTS5 ranking and safe search text, atomic ingest, CLI queries
Events and entities	aggregator, linker, build_events	evidence and duration, missed-frame tolerance, gap and zone splits, attribute compatibility, coarse linking, plate rules, people never linked, idempotent rebuilds, schema upgrades
Rules and alerts	rules, analyze	inclusive loitering boundary, after-hours window edges and midnight crossing, repeat vehicle, restricted zone, de-duplication, evidence chain, alert CLI
Emergencies and lifecycle	safety_rules, lifecycle	low battery warning, critical and hysteresis; degraded, lost and dropped-out signal; altitude envelope; geofence; escalation of the same alert; operator acknowledge and resolve; illegal transitions refused

Area	Modules	What is verified
Progressive runtime (dynamic inputs)	runtime, sampling, perception, cli_run, context_engine	tick ordering and pacing, alerts persisted while a run is in progress, streaming equals batch, sampling reasons, activity-only and count-only scene changes, perception timeout and fallback, the `run` command
Video and providers	video, video_ingest, cassette, gemini	unusable files, sampling plans, gate behaviour, carry-forward versus drop, cassette record/replay/miss/mismatch, provider-output validation, Gemini key handling, retries and error classification (SDK mocked)
Analyst	agent_core, agent_llm, agent_truncation, analyst_m12	bounded read-only tools, citation validation (nonexistent, unretrieved, malformed, unknown-type, missing, wording), retry then fallback, sessions, LangChain adapter with a fake chat model, truncation notices
Evaluation and repository rules	eval, ground_truth, sampler_noise_eval, scale_bench, scaffold	harness determinism, metric maths and zero denominators (F1 is 0, not N/A, when everything is wrong), ground-truth scoring, experiment mechanics, serialization, no network or key; core packages do not import LangChain
Dashboard	web_api, web_server, web_guard, web_bundle	read-only JSON API, localhost-only guard, static bundle, frame images

- **Offline and mocked boundaries.** No test needs a key or a network connection. The Gemini SDK is replaced by a fake client and the analyst model by a scripted stand-in or a fake chat model; a developer's real `.env` is never read by tests.
- **Regression strategy.** The shipped scenarios pin known facts (`security_events`: 7 events, 2 vehicle entities, 1 with several appearances, 7 alerts including a telemetry signal-loss alert; `normal_day`: none). Offline analyst answers recorded before the truncation fix must stay byte-identical when nothing is truncated. Mutation checks (deliberately breaking the code to see whether the tests notice) were used during development.
- **Evaluation tests** check that the harness computes what it claims and reproduces itself; they assert no timing thresholds and no tuned quality numbers.
- **Clean-clone verification.** The procedure is documented in `docs/testing.md`: clone into a fresh directory, create a virtual environment, `pip install -e ".[dev,agent]"`, run the tests and linter, then run the scripted demonstration and the evaluation; expect a clean git status because generated files are ignored. This report does not state its outcome; it is run and recorded at submission.
- **Not covered.** Live Gemini behaviour (live runs were separate, individually approved steps described in Section 15), perception accuracy, field performance, and the semantic truth of analyst prose.

13. Limitations and threats to validity

Documented limitations

Limitation	Consequence for the results
Synthetic and scripted evaluation	Fixtures are authored or generated, not real surveillance distributions; scripted detections are perfect, so vision-model perception accuracy is not measured. The rule metrics are agreement with labels written by the same author.
No ground-truth real-video labels	The single real clip has no annotations; zero alerts and 25 events are outputs of these settings on this clip, not quality scores. The clip is a traffic intersection, not fixed-property security footage, and is short (24.68 s).
Simulated telemetry, zone and time	All zone, time and telemetry values are assumptions supplied by the operator; any alert on real video is a consequence of them.
Frame gate is pixel-based	It is not semantic: at the default threshold it missed small meaningful changes on the fixture and accepted illumination-only changes; the default is uncalibrated.
Coarse entity matching	Can merge different vehicles with the same colour and body type and split one vehicle described inconsistently; never a proof of physical identity; people are not linked.
Event fragmentation	Free-text attributes and multi-object frames fragment events on real model output (20 of 25 single-frame events on the clip); loitering and repeat rules depend on event duration and appearance counts.
Citation validation is not entailment	It verifies ids, retrieval, presence and selected wording; all citation-valid but unsupported probe answers passed it. The eval-only structured checker covers only pre-structured claim classes. The validator was corrected against the adversarial suite, so that suite's accuracy is not independent evidence.
Small number of live Gemini runs	A few individually approved runs (Section 15) support no reliability or accuracy claim; one timed out at the 60 s limit and fell back, and one answer used inaccurate wording that still passed the gate.
One-machine scalability	Timings come from one machine and session on synthetic data, exclude vision-model latency, and show a superlinear entity-linking stage whose cause was not tested.

Limitation	Consequence for the results
Confidence is a placeholder	Stored confidence (0.5) is a constant, not a probability, and is not used for decisions.
Template analyst	Pattern-matched, not language understanding; the vehicle search only sees the first 20 entities the tool returns (and now says so).
Offline replay, not real time	"Real-time" behaviour is streaming aggregation on a simulated clock; no live feed was tested. Alerts are raised progressively only for scripted scenarios (`run`); a real video is ingested first, and its events and alerts are derived afterwards by `build-events` and `analyze`.
Loitering resolution	A loitering alert resolves when its event closes, including when the camera moves to another zone; the last frame in the zone still showed the person, so this is a simplification, not proof of departure.

14. Future work NOT IMPLEMENTED

None of the following exists in the repository; they are directions suggested by the measured limitations.

- **Stronger attribute normalisation** for free-text model output (for example canonical clothing colour and garment classes) so events fragment less; this needs its own evaluation because merging different people affects loitering.
- **Object tracking with bounding boxes** so that same-type objects in one frame can be told apart and event continuity is not inferred from text alone.
- **Improved entity matching**: time-gap and plausibility constraints, more distinctive attributes, and a faster linker (the measured superlinear stage).
- **Paging for analyst tools** so questions about large runs are answered completely instead of stating what was cut.
- **Calibrated vision confidence**, or a documented way to obtain it, and gate-threshold calibration on real footage.
- **Broader real-world evaluation**: fixed-property footage with human-annotated ground truth, longer clips, and more than one vision model.

15. AI-assisted development

Tool and role. Claude Code (Claude Sonnet 5.5) generated the code, tests, evaluation harness and documentation from my milestone specifications. Work proceeded one milestone at a time, each committed separately, and each entry in `docs/ai_usage.md` records what was generated, the design decisions and how the result was validated. My contribution was the requirements and scope cuts, the design decisions recorded in the log, and the corrections I directed. Gemini (`gemini-3.6-flash` in the recorded runs) is a runtime component (optional vision model and analyst backend), not a development assistant.

Milestone history

Milestone	Commit	Subject
M0	`d1c675d`	scaffold, feature spec, architecture plan (`.gitignore` first: `77330a6`)
M1	`ce851c9`	domain models, simulated telemetry, scripted provider, scenarios
M2	`67322ed`	SQLite persistence, FTS5 search, ingest and query CLI
M3	`f269130`	temporal events and coarse vehicle entity memory
M4	`fc2c4d3`	deterministic rules and evidence-backed alerts
M5A / M5B	`6c70037` / `8e7a3c4`	provider-neutral real-video pipeline and cassettes / Gemini frame describer
M6	`3883630`	grounded analyst with validated evidence citations
M7	`560c7c3` (tag `m7-final-baseline`)	evaluation, ablations and grounding analysis
M8a / M8b	`0ae7414` / `7589c2c`	explicit truncation in the offline analyst / real-video experiment documentation
M8d	`18bf196`, `61bc003`	documentation reconciliation; neutral placeholder test credentials
M9a / M9b / M9c	`e06b1d6` / `c93cc9a` / `e695bcd`	schema v4 for streaming runs / escalation ladders, drone-safety rules and alert lifecycle / scored, indexed vehicle re-identification
M10	`8af9457`	streaming patrol engine, adaptive sampling, perception agent, live view
M11	`33e2ad8`	ground-truth scenarios, alert-latency scoring, adaptive-sampling and noise experiments
M12	`465ab59`	hardened analyst: planner, replay, telemetry tools, per-sentence citation gate

Milestone	Commit	Subject
M13	`b3e81ac`	scale benchmark and bounded read paths (schema v5)
Dashboard	`5df9544` to `cffa515`	offline evaluator dashboard, then the frontend/backend layout
Audit pass	this revision	requirements-to-implementation audit against the assignment: scene-signature fix, prompt and tool-registry fix, F1 boundary fix, refreshed documentation, evaluation snapshot and this report

Validation approach and decisions

- **Validation.** Offline deterministic tests with mocked model boundaries; mutation checks that break the code to confirm the tests notice; live Gemini runs only as separate, individually approved steps; honest problem logs in each entry, including what did not work.
- **Decisions and corrections on record** include rejecting plan revisions until my scope cuts were reflected (M0), choosing carry-forward for gate-skipped frames (M5A), reporting the gate and citation-suite results without tuning them (M7), fixing two citation-validator behaviours after review while stating that this makes the suite's accuracy non-independent (M7), and preserving the entity-linking scaling result as a measured baseline instead of optimising it.
- **Live Gemini observations (not evaluation).** Recorded live runs produced grounded analyst answers with verified citations; one run hit a transient read timeout at the 60 s limit and the deterministic fallback answered; a later run of the same question succeeded in 13.02 s with 3 model rounds, 4 tool calls (4 unique tools), 0 correction attempts and no fallback. One live answer used the phrase "vehicle tracking", which the system does not do, and passed the gate. This small sample supports no reliability claim.
- **Accidental live verification during the final documentation pass.** While checking the README's demonstration commands I ran two Gemini-backed commands (one analyst query and one frame-description operation) intending to confirm the missing-key error message. The locally configured `.env` supplied a key, so they executed as live calls. The exact number of API requests is unknown and was not recorded. Neither operation writes output; no tracked repository file and no real-video artifact was modified, and no further live calls were made. They were an accidental verification of the error path, not part of any experiment, and **no result from them is used anywhere in this report or the project**.
- **Secrets.** The API key lives only in a local, gitignored `.env` and the process environment; it appears in no committed file, and test credentials are neutral placeholders.

16. Demo and submission materials

The demonstration videos are screen recordings with the author's voiceover. They use offline commands and cassette replay; no live Gemini call is required. The video links are provided with the submission.

Planned demonstration videos

Video	Shows
1. Scripted end to end	Ingesting the `security_events` scenario into the SQLite index; queries by time, zone, object and keyword; building events and entities; running the rules; an alert traced to its evidence frames; analyst questions with citations; a run summary and a follow-up question.
2. Real video with cassette replay	The real clip; the recorded Gemini descriptions replayed with no API call; the gate and sampling summary; events, entities and the zero-alert result with the reasons; the analyst's truncation notices; the stated limitations.
3. Evaluation and scalability	The `evaluate` command and ablation table; the gate's misses; flicker and entity-memory ablations; the citation suite and the semantic probe; the scale benchmark with its environment.
4. Design and testing (optional)	Architecture walk-through, the test suite and linter, and the AI-usage log.

The submission consists of the private GitHub repository (code, documentation and design artifacts, to be shared with `assignments@flytbase.com` as a contributor), this PDF report, and the video links with view access.

17. Conclusion

The prototype implements the requested pipeline end to end: simulated telemetry and video frames become indexed observations, temporal events, coarse vehicle entities and evidence-linked alerts from eight deterministic rules (raised progressively for a simulated patrol), and an analyst answers and summarises with citations that a validator checks against the

database. A real vision model (Gemini) integrates through a provider contract with cassette record and replay, and one real clip ran through the entire pipeline.

The evaluation shows what each component contributes and where it fails: the frame gate saves calls but is pixel-based and missed small meaningful changes on the fixture; missed-frame tolerance prevents one undetected frame from removing a loitering alert; entity memory enables the repeat rule but can link different vehicles; the rules agree with authored labels on controlled scenarios; and the citation gate rejects the categories it checks while accepting every citation-valid but unsupported answer in the probe. The real-video run validates integration and exposes event fragmentation, inconsistent model attributes and analyst truncation, one of which was fixed.

These results are measurements on synthetic fixtures, one short real clip and one machine. They do not establish real-world perception accuracy, security-detection performance, entity identity, semantic faithfulness of analyst prose, or production-scale behaviour, and the report makes no such claims.

Appendix A. Assignment requirement coverage

Assignment item	Where it is met
Process simulated telemetry and video frames	Sections 2, 3, 5; `simulate`, `ingest` (scripted frames with seeded telemetry) and `ingest-video` (real video)
Identify objects/events and log them with context	Sections 4-6; observations with zone, time and telemetry; events with duration; Gemini descriptions in Section 10
Alerts from predefined rules (immediate alerts)	Section 7; `run` (progressive), `analyze`, `alerts --show`; emergency scenarios in Section 9.9
Cross-domain frame-by-frame indexing	Section 4; SQLite `frames` / `observations` with FTS5, queried by time, zone, object or keyword
Feature spec (value, 2-3 requirements)	`docs/feature_spec.md`
Architecture and design artifacts	Section 3 and Figures 1-3; `docs/architecture.md`
Use AI to generate components; QA test cases	Sections 12 and 15; `docs/testing.md`, `docs/ai_usage.md`
Expected output: logs, alert, queryable frames	Scripted demonstration (video 1): e.g. the loitering alert "Person stayed in main_gate for 480s, reaching the 300s loitering threshold", and `query --search` / `--object` over indexed frames
Bonus: video summary and follow-up questions	Section 8.3; `summarize`, `ask --follow-up`, `--interactive`

Source documents in the repository: `README.md`, `docs/feature_spec.md`, `docs/architecture.md`, `docs/evaluation.md`, `docs/real_video_experiment.md`, `docs/testing.md`, `docs/ai_usage.md`.